

SIKSHA ‘O’ ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with A++ Grade

Admission Batch: 2023 | Session: 2025-26

Major Project Report of

MACHINE LEARNING CONCEPT-2 (CSE 3968)

On

MLC Network Detection System

Network Anomaly Detection Using Machine Learning

IoT-23 Dataset | Bagging | Boosting | Random Forest | Isolation Forest | One-Class SVM | Autoencoder |
Streamlit

Submitted by:

Name	Registration Number
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Section: 2341_____ Semester: _____ Branch: Computer Science & Engineering

SOA LOGO

Department of Computer Science & Engineering

Institute of Technical Education and Research (ITER), S'O'A
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

May, 2026

CERTIFICATE

This is to certify that the Major Project Report titled “**MLC Network Detection System: Network Anomaly Detection Using Machine Learning**” submitted by the students listed below has been carried out under the supervision of the undersigned and is a bonafide record of work done in partial fulfilment of the requirements of **Machine Learning Concept-2 (CSE 3968)**, academic session 2025–26.

Name	Registration No.
NAME 1	2341_____
NAME 2	2341_____
NAME 3	2341_____
NAME 4	2341_____

Supervisor / Faculty Guide

Name: _____

Designation: _____

Dept: CSE, ITER, S'O'A

Head of Department

Name: _____

Designation: Professor & HOD

Dept: CSE, ITER, S'O'A

Date: _____**Place:** Bhubaneswar, Odisha

ABSTRACT

The **MLC Network Detection System** is a machine learning-based network anomaly detection framework designed to classify network traffic as *Benign* or *Malicious*. Traditional signature-based Intrusion Detection Systems are effective for known attacks but fail when attack behavior changes or when zero-day threats appear. This project uses machine learning to learn traffic behavior from the **IoT-23 dataset**, a real-world collection of benign and malicious IoT network connection logs.

The pipeline includes dataset loading, data cleaning, Exploratory Data Analysis, feature engineering, preprocessing, model training, evaluation, model saving using `.pkl`, and Streamlit web deployment. A sample of **50,000 connection records** was used. Multiple models were trained and compared, including Bagging, Boosting, Isolation Forest, One-Class SVM, Autoencoder, and Random Forest.

Key Results: Random Forest achieved Accuracy 0.9979, Precision 0.9962, Recall 1.0000, and F1-Score 0.9981. The saved model was integrated into a Streamlit app that supports manual traffic prediction and row-wise prediction from `testing_data.csv`.

Keywords: Network Anomaly Detection, IoT-23, Random Forest, Isolation Forest, One-Class SVM, Autoencoder, Ensemble Learning, Streamlit, Intrusion Detection, Feature Engineering.

Contents

1	Introduction	7
1.1	Background	7
1.2	Project Overview	7
1.3	Why IoT-23 Dataset?	7
2	Problem Statement	7
3	Objectives	7
4	Dataset Description and Understanding	8
4.1	Dataset Used	8
4.2	Dataset Columns	8
4.3	Target Variable	8
5	Data Preprocessing	9
5.1	Preprocessing Need	9
5.2	Preprocessing Steps	9
5.3	Train-Test Split	9
6	Exploratory Data Analysis	9
6.1	EDA Purpose	9
6.2	Class Distribution	10
6.3	Numeric Feature Distributions	11
6.4	Correlation Heatmap	12
6.5	EDA Intuition	12
7	Feature Engineering	13
7.1	Why Feature Engineering Was Required	13
7.2	Feature Importance	14
8	Implementation	14
8.1	Implementation Steps	14
8.2	Technology Stack	15
8.3	System Architecture	16
8.4	Streamlit Workflow	17
9	Model Development and Justification	17
9.1	Model Selection Philosophy	17
9.2	Models Used	17
9.3	Bagging Models	18
9.4	Boosting Models	18
9.5	Unsupervised Anomaly Detection	18
9.6	Deep Learning: Autoencoder	18
9.7	Why Bagging and Boosting Performed Well	18
9.8	Why Random Forest Performed Better	19
9.9	Why Other Models Were Lower	19
9.10	EDA Intuition Connected with Model Performance	19
10	Evaluation Metrics	20

11 Experimental Results	20
11.1 Bagging and Boosting Results	20
11.2 Independent Model Results	21
12 Streamlit Web Application	22
12.1 Application Overview	23
12.2 Implemented App Features	23
12.3 How to Run	23
13 Applications	26
14 Challenges and Limitations	27
14.1 Technical Challenges	27
14.2 Limitations	27
15 Future Scope	27
16 Conclusion	28
References	28
Appendix: Source Code Listings	30

List of Figures

1	Benign vs Malicious traffic distribution in testing data.	10
2	Distribution of key numeric traffic features using log transformation.	11
3	Correlation heatmap of network traffic features.	12
4	Top feature importance values from the saved Random Forest model.	14
5	System architecture from dataset loading to Streamlit deployment.	16
6	Streamlit workflow for manual input and testing-data row prediction.	17
7	Bagging and Boosting model accuracy comparison.	21
8	Independent model comparison across accuracy, precision, recall, and F1-score.	22
9	Placeholder for Streamlit home page screenshot. Replace with actual screenshot.	24
10	Placeholder for manual prediction screenshot. Replace with actual screenshot.	25
11	Placeholder for testing-data row prediction screenshot. Replace with actual screenshot.	26

List of Tables

1	IoT-23 Zeek Connection Log Column Description	8
2	Preprocessing Steps and Justification	9
3	Engineered Features and Their Security Importance	13
4	Technology Stack Used	15
5	Models Used and Their Justification	18
6	How EDA Findings Explain Model Performance	19
7	Evaluation Metrics and Importance	20
8	Bagging and Boosting Model Results	20
9	Interpretation of Bagging and Boosting Results	20
10	Independent Model Results	21
11	Interpretation of Independent Model Results	22
12	Project Files and Their Purpose	30

1. Introduction

1.1 Background

Cybersecurity is one of the most important areas in modern computing. As IoT devices, cloud platforms, smart infrastructure, and enterprise networks grow, the number of cyberattacks also increases. Attackers use port scanning, malware communication, botnets, denial-of-service attacks, unauthorized access attempts, and command-and-control communication to compromise systems.

Traditional Intrusion Detection Systems depend heavily on predefined signatures. These systems detect known attacks, but they are weak against unknown or zero-day attacks. Machine learning can improve this by learning patterns from historical network traffic and detecting unusual behavior.

1.2 Project Overview

The **MLC Network Detection System** is an end-to-end project that trains machine learning models on IoT network traffic and deploys the best model through a Streamlit web application. The model predicts whether a connection is **Benign** or **Malicious**.

1.3 Why IoT-23 Dataset?

The IoT-23 dataset contains real IoT malware and benign traffic scenarios. It is more suitable for modern IoT security research than older benchmark datasets because it includes realistic Zeek/Bro network logs and real malware family behavior.

2. Problem Statement

Modern networks generate large volumes of connection logs. Manually analyzing every network record is impossible. Traditional IDS tools face the following issues:

- They depend on known signatures.
- They cannot reliably detect zero-day attacks.
- They require frequent manual rule updates.
- They may generate false positives.
- They struggle with large-scale IoT traffic.
- Malware behavior changes over time.

Core Problem: Build a machine learning system that learns from IoT network traffic and classifies a connection as **Benign** or **Malicious**, then deploy the saved model through an interactive Streamlit web application.

3. Objectives

The objectives of this project are correct and aligned with the implemented system:

1. Load and understand the IoT-23 network traffic dataset.
2. Use a 50,000-row sample for efficient experimentation.
3. Clean missing, invalid, and mixed-type values in Zeek logs.
4. Perform EDA to understand traffic patterns, class distribution, feature distribution, and correlation.
5. Engineer network-behavior features such as total bytes, total packets, byte ratios, packet ratios,

- missing-value indicators, and port behavior.
6. Train and compare Bagging, Boosting, Random Forest, Isolation Forest, One-Class SVM, and Autoencoder models.
 7. Evaluate models using Accuracy, Precision, Recall, F1-score, and Confusion Matrix.
 8. Save the best-performing model as a reusable `.pkl` file.
 9. Build a Streamlit application for manual prediction and test-data row prediction.
 10. Visualize model output and traffic properties in the application.

4. Dataset Description and Understanding

4.1 Dataset Used

The project uses the **IoT-23 Dataset**, which contains real benign and malicious IoT traffic. The notebook uses the Zeek/Bro connection log file:

```
opt/Malware-Project/BigDataset/IoTScenarios/
CTU-IoT-Malware-Capture-1-1/bro/conn.log.labeled
```

Only 50,000 rows were loaded to make training practical:

```
1 SAMPLE_SIZE = 50_000
```

4.2 Dataset Columns

Table 1: IoT-23 Zeek Connection Log Column Description

Column	Meaning
ts	Unix timestamp of the network connection.
uid	Unique connection ID. It is useful for logging but removed during model training.
id.orig_h	Origin/source IP address.
id.orig_p	Origin/source port number.
id.resp_h	Response/destination IP address.
id.resp_p	Response/destination port number.
proto	Transport protocol such as TCP, UDP, or ICMP.
service	Application service such as HTTP, DNS, SSH, or missing service.
duration	Duration of the connection.
orig_bytes	Bytes sent by origin host.
resp_bytes	Bytes sent by response host.
conn_state	Zeek connection state, such as S0, SF, REJ, RSTO.
missed_bytes	Bytes missed during capture.
history	Packet history pattern.
orig_pkts	Packets sent by origin host.
resp_pkts	Packets sent by response host.
label	Raw traffic label containing benign or malicious information.

4.3 Target Variable

The raw `label` column was converted into a clean binary target:

Benign = normal traffic

Malicious = attack/anomaly traffic

5. Data Preprocessing

5.1 Preprocessing Need

The dataset contains missing values represented by symbols such as `-`, empty strings, and `(empty)`. Some numeric columns are read as object type because of mixed values. Preprocessing is necessary so the models receive clean, numeric, and consistent input.

5.2 Preprocessing Steps

Table 2: Preprocessing Steps and Justification

Step	Applied To	Reason
Missing value conversion	Raw Zeek fields	Converts <code>-</code> , blank values, and <code>(empty)</code> into proper missing values.
Numeric coercion	Byte, packet, duration, and port fields	Ensures mathematical operations and model training can be performed correctly.
Median imputation	Numeric columns	Handles missing numeric values while reducing outlier effect.
Most frequent imputation	Categorical columns	Fills missing category values using the most common class.
One-hot encoding	Categorical columns	Converts protocol, service, state, history, and prefix values into model-readable numeric columns.
Standard scaling	Numeric columns	Improves model stability, especially for SVM and Autoencoder.
Train-test split	Full dataset	Splits 80% for training and 20% for testing using stratification.

5.3 Train-Test Split

The dataset was split 80/20 with stratification (Code 6, Appendix):

- **Training set:** 40,000 records (80%)
- **Test set:** 10,000 records (20%)

6. Exploratory Data Analysis

6.1 EDA Purpose

EDA was performed to understand class distribution, missing values, numeric feature behavior, and relationships between features. This is important because network attack patterns are often visible through abnormal byte count, packet count, connection duration, missing response, or unusual port behavior.

6.2 Class Distribution

Figure 1: Benign vs Malicious traffic distribution in testing data.

The class distribution helps us understand whether the model could be biased toward one class. In network security, even if the dataset is moderately imbalanced, recall remains important because missing malicious traffic is risky.

6.3 Numeric Feature Distributions

Figure 2: Distribution of key numeric traffic features using log transformation.

Network traffic values are usually skewed. For example, most connections may transfer very few bytes, while a few connections transfer large amounts. Log transformation helps reveal the hidden distribution pattern.

6.4 Correlation Heatmap

Figure 3: Correlation heatmap of network traffic features.

The heatmap shows that packet-related and byte-related features are related. This supports the idea that engineered features such as total packets, total bytes, and byte-per-packet ratio can improve model performance.

6.5 EDA Intuition

EDA Intuition:

- Malicious scanning traffic often has low duration, low bytes, and repeated failed connection states.
- Malware communication may target specific destination ports repeatedly.
- Missing service or response bytes can indicate incomplete, rejected, or suspicious connections.
- Packet and byte ratios help detect asymmetric communication such as one-way scanning or command-and-control behavior.

7. Feature Engineering

7.1 Why Feature Engineering Was Required

Raw network fields do not always directly show attack behavior. Feature engineering converts raw columns into behavioral signals. For example, `orig_bytes` and `resp_bytes` separately give volume information, but `orig_resp_byte_ratio` explains communication imbalance.

Table 3: Engineered Features and Their Security Importance

Feature	Meaning	Why It Helps
<code>service_missing</code>	Whether service is missing	Missing service often occurs in failed scans or incomplete connections.
<code>history_missing</code>	Whether packet history is missing	Incomplete history may indicate abnormal capture or traffic behavior.
<code>orig_bytes_missing</code>	Missing origin bytes flag	May indicate failed or one-sided traffic.
<code>resp_bytes_missing</code>	Missing response bytes flag	No response can occur during scanning or blocked malicious attempts.
<code>hour</code>	Hour extracted from timestamp	Captures time-based attack behavior.
<code>dayofweek</code>	Day of week from timestamp	Captures periodic traffic behavior.
<code>total_bytes</code>	Origin + response bytes	Measures total connection data volume.
<code>total_pkts</code>	Origin + response packets	Measures total packet activity.
<code>bytes_per_pkt</code>	Average packet size	Detects abnormal packet-size patterns.
<code>orig_resp_byte_ratio</code>	Origin/response byte ratio	Detects asymmetric communication and possible scanning.
<code>orig_resp_pkt_ratio</code>	Origin/response packet ratio	Detects packet-level imbalance.
<code>resp_port_is_well_known</code>	Destination port ≤ 1024	Identifies common attack targets such as SSH, Telnet, HTTP.
<code>orig_ip_prefix</code>	Source network prefix	Captures subnet-level behavior without using full IP.
<code>resp_ip_prefix</code>	Destination network prefix	Captures target network pattern.

7.2 Feature Importance

Figure 4: Top feature importance values from the saved Random Forest model.

The feature importance graph shows that the Random Forest model relies strongly on network behavior indicators such as port-related features, connection state, packet count, and byte count. This aligns with the EDA intuition that attacks often differ from normal traffic through communication imbalance, port behavior, and missing or failed responses.

8. Implementation

8.1 Implementation Steps

1. **Dataset Loading:** Loaded the IoT-23 connection log using Pandas with explicit column names and missing-value indicators.
2. **Target Cleaning:** Converted raw labels into **Benign** and **Malicious**.
3. **EDA:** Generated class distribution, numeric distribution, and correlation heatmap.
4. **Feature Engineering:** Created missing flags, time features, byte/packet aggregates, traffic ratios, port flags, and IP prefixes.
5. **Train-Test Split:** Used 80/20 stratified split to preserve label distribution.

6. **Preprocessing Pipeline:** Used `ColumnTransformer` for numeric and categorical preprocessing.
7. **Model Training:** Trained Bagging, Boosting, Random Forest, Isolation Forest, One-Class SVM, and Autoencoder models.
8. **Evaluation:** Compared models using accuracy, precision, recall, F1-score, and confusion matrix.
9. **Model Saving:** Saved the best Random Forest pipeline as `best_network_anomaly_model.pkl`.
10. **Streamlit Deployment:** Built `app.py` to load the saved model and provide manual and CSV-row prediction.

8.2 Technology Stack

Table 4: Technology Stack Used

Category	Technology
Programming Language	Python
Notebook Environment	Jupyter Notebook / Google Colab
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	Scikit-learn
Deep Learning	TensorFlow / Keras
Model Persistence	Joblib, Pickle
Web Application	Streamlit
Dataset	IoT-23 Dataset

8.3 System Architecture

Figure 5: System architecture from dataset loading to Streamlit deployment.

8.4 Streamlit Workflow

Figure 6: Streamlit workflow for manual input and testing-data row prediction.

9. Model Development and Justification

9.1 Model Selection Philosophy

This project trains both *supervised* and *unsupervised* models. Supervised models (Random Forest, Bagging, Boosting) use labeled data and achieve high accuracy on known attack patterns. Unsupervised anomaly detection models (Isolation Forest, One-Class SVM, Autoencoder) learn only from normal traffic and can detect unknown attacks. Code 7 (Appendix) implements the Bagging/Boosting training. Code 9 (Appendix) implements the anomaly detection models.

9.2 Models Used

Table 5: Models Used and Their Justification

Model	Type	Why It Was Used
Bagging Decision Tree	Supervised ensemble	Reduces variance of a single decision tree by training many trees on bootstrap samples.
Random Forest	Supervised ensemble	Handles tabular data very well, captures non-linear patterns, reduces overfitting, and provides feature importance.
AdaBoost	Boosting ensemble	Focuses on misclassified samples and improves weak learners.
Gradient Boosting	Boosting ensemble	Builds models sequentially to correct previous errors.
Hist Gradient Boosting	Boosting ensemble	Efficient for larger datasets using histogram-based binning.
Isolation Forest	Unsupervised anomaly detection	Isolates unusual records quickly and is useful when attack labels are unavailable.
One-Class SVM	Unsupervised anomaly detection	Learns the boundary of normal traffic and flags deviations.
Autoencoder	Deep learning anomaly detection	Learns compressed normal traffic representation and detects high reconstruction error.

9.3 Bagging Models

Bagging with Decision Tree: Multiple trees trained on bootstrap samples; final prediction by majority vote. Reduces variance compared to a single tree. **Random Forest:** Extends bagging by additionally randomizing feature selection at each split – best-in-class for tabular data [?].

9.4 Boosting Models

AdaBoost [?]: Sequentially re-weights misclassified samples. **Gradient Boosting** [?]: Each tree fits the negative gradient of the loss function – extremely powerful for structured data. **HistGradientBoosting:** Bins continuous features into histograms for significantly faster training on 50,000+ records.

9.5 Unsupervised Anomaly Detection

Isolation Forest [?]: Randomly selects a feature and split point. Anomalies are isolated in fewer splits because they are rare and different. **One-Class SVM** [?]: Finds a hyperplane in kernel space separating normal data from the origin – test points beyond this boundary are flagged as anomalies.

9.6 Deep Learning: Autoencoder

Architecture: Input $\rightarrow$ Dense(64, relu) $\rightarrow$ Dense(32, relu) $\rightarrow$ Dense(16, relu) $\rightarrow$ Dense(32, relu) $\rightarrow$ Dense(64, relu) $\rightarrow$ Output. Trained on *benign traffic only* using MSE loss with EarlyStopping [?]. Reconstruction error threshold at the 95th percentile separates benign from malicious at inference time.

9.7 Why Bagging and Boosting Performed Well

Bagging and Boosting performed well because the dataset contains structured tabular signals. From EDA, we observed that malicious traffic differs through connection state, port behavior, byte/packet counts, and missing service or response values. Tree-based ensemble models are strong at capturing such non-linear thresholds. For example:

- If destination port is suspicious and response bytes are missing, the traffic may be malicious.
- If connection state is failed and packet count is low, it may indicate scanning.
- If byte/packet ratios are unusual, it may indicate abnormal communication.

Decision-tree-based ensembles naturally split data using such rules. Bagging improves stability by reducing variance, while Boosting improves accuracy by focusing on difficult samples.

9.8 Why Random Forest Performed Better

Random Forest performed better because it aligns strongly with the EDA findings:

- EDA showed that traffic behavior is rule-like: ports, states, missing values, packets, and bytes separate many benign and malicious cases.
- Random Forest combines many decision trees, so it learns multiple traffic patterns instead of depending on one rule.
- It handles non-linear relationships such as byte ratio plus connection state plus destination port.
- It is robust against noise and mixed feature types after preprocessing.
- It provides feature importance, which confirmed that important network-behavior features influenced predictions.

9.9 Why Other Models Were Lower

- **Isolation Forest:** It detected all malicious samples with high recall, but it also marked many benign records as malicious. This reduced precision.
- **One-Class SVM:** It learned the normal boundary, but network traffic is complex and overlapping. It needs careful kernel and `nu` tuning.
- **Autoencoder:** It had good precision but lower recall. Better hidden-layer design, more epochs, threshold tuning, and training on more benign traffic could improve it.
- **AdaBoost:** It performed well but slightly lower than Gradient Boosting because the dataset may contain patterns better handled by deeper tree ensembles.

9.10 EDA Intuition Connected with Model Performance

The EDA showed that malicious and benign traffic differ through interpretable network-behavior signals. These signals are not purely linear; they appear as combinations such as destination port plus connection state plus response-byte behavior. This explains why tree-based ensemble models performed better.

Table 6: How EDA Findings Explain Model Performance

EDA Finding	Security Meaning	Model Impact
Low bytes and low packets	Common in scans, failed probes, or blocked malicious attempts	Decision-tree models split well on thresholds such as zero response bytes and small packet counts.
Connection state patterns	States such as S0 or failed connections can indicate incomplete or suspicious traffic	Random Forest and Gradient Boosting learn state-specific branches effectively.
Port behavior	IoT attacks often target services such as Telnet, SSH, HTTP, and uncommon exposed ports	Port-based features support strong tree splits and improve ensemble separation.
Missing service/response fields	Missing values often indicate incomplete or refused traffic	Missing-value flags became useful engineered features for Random Forest.
Non-linear ratios	Byte and packet ratios describe asymmetric communication	Tree ensembles capture ratio thresholds better than simple linear boundary models.

Main Interpretation: Random Forest performed better because its structure is aligned with the

data intuition from EDA. The dataset contains many rule-like patterns: if a connection has a suspicious destination port, missing response bytes, failed connection state, and abnormal packet ratio, it becomes easier to separate malicious traffic. Random Forest combines many such decision rules across many trees, which gives high accuracy and strong recall.

10. Evaluation Metrics

Table 7: Evaluation Metrics and Importance

Metric	Meaning	Importance in Network Security
Accuracy	Correct predictions divided by total predictions.	Shows overall performance but may be misleading if classes are imbalanced.
Precision	Correct malicious predictions divided by all predicted malicious records.	Reduces false alarms and analyst workload.
Recall	Correct malicious predictions divided by all actual malicious records.	Very important because missing a real attack is dangerous.
F1-score	Balance between precision and recall.	Useful when both false alarms and missed attacks matter.
Confusion Matrix	Shows TP, TN, FP, and FN.	Explains where the model is making mistakes.

11. Experimental Results

11.1 Bagging and Boosting Results

Table 8: Bagging and Boosting Model Results

Rank	Model	Accuracy	Precision	Recall	F1-Score
1	Bagging – Decision Tree	1.0000	1.00	1.00	1.00
2	Boosting – Gradient Boosting	1.0000	1.00	1.00	1.00
3	Boosting – Hist Gradient Boosting	0.9987	1.00	1.00	1.00
4	Bagging – Random Forest	0.9979	1.00	1.00	1.00
5	Boosting – AdaBoost	0.9920	0.99	0.99	0.99

Table 9: Interpretation of Bagging and Boosting Results

Model	Interpretation
Bagging – Decision Tree	Captured rule-like separation in port, byte, packet, and connection-state features. The result matches EDA findings where malicious traffic showed clear behavioral differences.
Boosting – Gradient Boosting	Sequentially corrected difficult samples and learned fine-grained traffic boundaries, especially around byte/packet ratios and connection states.
Boosting – Hist Gradient Boosting	Performed almost perfectly while being efficient for structured tabular data because histogram binning works well with numeric traffic fields.
Bagging – Random Forest	Stable ensemble performance; slight variation can occur because Random Forest averages many trees and generalizes instead of memorizing a single split.
Boosting – AdaBoost	Strong performance, but slightly more sensitive to noisy or borderline samples because it repeatedly increases weight on misclassified points.

Figure 7: Bagging and Boosting model accuracy comparison.

11.2 Independent Model Results

Table 10: Independent Model Results

Rank	Model	Accuracy	Precision	Recall	F1-Score
1	Random Forest	0.9979	0.9962	1.0000	0.9981
2	Isolation Forest	0.7797	0.7168	1.0000	0.8350
3	One-Class SVM	0.7771	0.7144	1.0000	0.8334
4	Autoencoder	0.8158	0.9462	0.7100	0.8113

Table 11: Interpretation of Independent Model Results

Model	Interpretation
Random Forest	Best overall. It matched the EDA intuition because tree splits captured port behavior, byte/-packet volume, connection-state patterns, and missing-value indicators.
Isolation Forest	Detected all malicious traffic, giving perfect recall, but produced more false positives because it treats many unusual benign records as anomalies.
One-Class SVM	Also achieved perfect recall, but lower precision. Better results may require careful tuning of kernel, nu, gamma, and normal-sample selection.
Autoencoder	High precision but lower recall. It may need better hidden-layer depth, threshold selection, regularization, more epochs, and more benign-only training data.

Figure 8: Independent model comparison across accuracy, precision, recall, and F1-score.

Best Final Model: Random Forest			
Accuracy	Precision	Recall	F1-Score
0.9979	0.9962	1.0000	0.9981
Saved as: best_network_anomaly_model.pkl			

12. Streamlit Web Application

12.1 Application Overview

Streamlit was used to convert the trained machine learning model into a usable web application. The application loads the saved .pkl model and allows users to predict whether traffic is benign or malicious.

12.2 Implemented App Features

- **Model loading:** Loads `best_network_anomaly_model.pkl` using Joblib.
- **Manual prediction:** User enters one network traffic record through a form.
- **testing_data.csv prediction:** User selects one row from testing data and compares actual vs predicted label.
- **Feature preparation:** Uses the same feature engineering logic as the notebook.
- **Prediction display:** Shows Benign or Malicious result.
- **Confidence chart:** Shows prediction probability when supported by the model.
- **Traffic visualizations:** Displays byte volume, packet volume, and port summary.
- **Sidebar:** Shows model status, project information, and label guide.
- **Error handling:** Displays readable messages for model-loading or version mismatch errors.

12.3 How to Run

```
cd D:\MLC_Projects
python -m streamlit run app.py
```

Figure 9: Placeholder for Streamlit home page screenshot. Replace with actual screenshot.

Figure 10: Placeholder for manual prediction screenshot. Replace with actual screenshot.

Figure 11: Placeholder for testing-data row prediction screenshot. Replace with actual screenshot.

13. Applications

- **Enterprise Network Monitoring:** The model can act as a first-level detector for suspicious connections in organizational traffic.
- **IoT Device Protection:** IoT devices are common malware targets; this system can help identify botnet-like communication and scanning behavior.
- **Smart City Infrastructure:** Traffic from cameras, sensors, and gateways can be monitored for abnormal patterns.
- **Cloud Traffic Anomaly Detection:** Cloud network logs can be processed to detect lateral movement and suspicious service access.
- **Cybersecurity Education:** The project demonstrates the complete ML workflow from dataset to deployed app.
- **Preliminary IDS Dashboard:** The Streamlit app can be extended as a dashboard for analysts to inspect suspicious records.
- **Research Baseline:** Results can be compared with future models, larger datasets, or new feature engineering methods.

14. Challenges and Limitations

14.1 Technical Challenges

- **Mixed data types:** Zeek logs use - for missing values, requiring explicit `na_values` during loading.
- **Binary label format for anomaly models:** Isolation Forest and One-Class SVM output +1/-1; Autoencoder outputs reconstruction error. These required careful threshold conversion to match the Benign/Malicious labels.
- **Autoencoder threshold selection:** The reconstruction error percentile threshold directly impacts Precision-Recall trade-off.
- **Scikit-learn version dependency:** The `.pkl` pipeline requires the same Scikit-learn version used during training; version mismatches cause Streamlit loading errors.
- **One-Class SVM training time:** $O(n^2)$ complexity required subsampling to 10,000 benign records.

14.2 Limitations

- Only 50,000 of the available millions of IoT-23 records were used.
- The model is trained on IoT-23 Scenario 1; generalization to other malware families may require retraining.
- Real-time packet capture is not implemented; the app works on pre-recorded connection logs.
- The current system predicts one record at a time; batch CSV upload is not available.

15. Future Scope

- Add batch CSV upload prediction in Streamlit so many network records can be tested together.
- Integrate real-time packet capture using tools such as Zeek, Scapy, Tshark, or Wireshark exports.
- Simulate real-life attacks in a safe lab environment using tools such as Nmap, Hping3, Metasploit test machines, and Kali Linux. The captured traffic can then be converted into Zeek-style logs and tested against the trained model.
- Test the model against real-life attack categories such as port scanning, brute-force attempts, denial-of-service traffic, malware command-and-control communication, and suspicious IoT device behavior.
- Train on more IoT-23 scenarios and additional datasets such as CICIDS or NSL-KDD for broader generalization.
- Add explainable AI using SHAP or LIME to show why a record was classified as malicious.
- Add alerting through email, dashboard notifications, or SIEM integration.
- Deploy the Streamlit app on Streamlit Community Cloud or a cloud server.
- Optimize Autoencoder architecture using more hidden layers, dropout, regularization, and threshold tuning.
- Add continuous retraining when new labeled traffic becomes available.

16. Conclusion

The **MLC Network Detection System** successfully demonstrates a complete machine learning workflow for network anomaly detection. The project uses IoT-23 traffic data, performs cleaning and EDA, creates meaningful engineered features, trains multiple models, evaluates them with proper metrics, saves the best model, and deploys it using Streamlit.

Random Forest was selected as the final model because it achieved the best independent result with Accuracy 0.9979, Precision 0.9962, Recall 1.0000, and F1-score 0.9981. This strong performance is aligned with the EDA intuition: the dataset contains rule-like patterns in ports, packet counts, byte ratios, connection state, and missing-value indicators, which tree-based ensembles capture effectively.

The final Streamlit application makes the system interactive and practical by supporting both manual prediction and prediction from `testing_data.csv`. This project shows how machine learning can support adaptive cybersecurity and intelligent network monitoring.

References

- [1] S. García, M. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Zenodo, 2020. <https://doi.org/10.5281/zenodo.4743746>
- [2] Stratosphere Laboratory, “IoT-23 Dataset,” Czech Technical University. <https://www.stratosphereips.org/dataset>
- [3] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. <https://doi.org/10.1023/A:1010912341470>
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” *IEEE ICDM*, 2008. <https://doi.org/10.1109/ICDM.2008.4717973>
- [5] B. Schölkopf et al., “Estimating the Support of a High-Dimensional Distribution,” *Neural Computation*, 2001. <https://doi.org/10.1162/089976601750264965>
- [6] M. Sakurada and T. Yairi, “Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction,” *MLSDA*, 2014. <https://doi.org/10.1145/2689746.2689747>
- [7] Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- [8] Streamlit Documentation. <https://docs.streamlit.io/>
- [9] Pandas Documentation. <https://pandas.pydata.org/docs/>
- [10] TensorFlow Documentation. <https://www.tensorflow.org/>
- [11] S. García, M. Parmisano, and M. J. Erquiaga, “IoT-23: A labeled dataset with malicious and benign IoT network traffic,” Zenodo, 2020. <https://doi.org/10.5281/zenodo.4743746>
- [12] S. García et al., “IoT-23 Dataset,” Stratosphere Laboratory, CTU, Prague, 2020.
- [13] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. 2nd IEEE Symp. Comput. Intell. Security Def. Appl.*, 2009, pp. 1–6.
- [14] D. E. Denning, “An intrusion-detection model,” *IEEE Trans. Softw. Eng.*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.
- [15] J. McHugh, “Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA evaluations,” *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 4, pp. 262–294, Nov. 2000.
- [16] L. Breiman, “Random forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [17] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [18] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [19] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

- [20] B. Schölkopf et al., “Estimating the support of a high-dimensional distribution,” *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [21] M. Sakurada and T. Yairi, “Anomaly detection using autoencoders with nonlinear dimensionality reduction,” in *Proc. MLSDA Workshop ACM SIGKDD*, 2014.
- [22] M. Ring et al., “A survey of network-based intrusion detection data sets,” *Comput. Secur.*, vol. 86, pp. 147–167, Sep. 2019.
- [23] Streamlit Inc., “Streamlit documentation,” 2024. <https://docs.streamlit.io/>
- [24] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. <https://scikit-learn.org/>

Appendix: Important Code Snippets

All code is from MLC_NetWordDetection_System.ipynb. Code is referenced in-text by label (e.g., Code 2).

Table 12: Project Files and Their Purpose

File	Purpose
MLC_NetWordDetection_System.ipynb	Jupyter Notebook – complete training pipeline
best_network_anomaly_model.pkl	Serialized best model (Random Forest pipeline)
best_iot23_ensemble_model.joblib	Joblib format of saved model
model_results.csv	Model comparison metrics CSV
app.py	Streamlit web application
testing_data.csv	Test records for Streamlit row-prediction mode
requirements.txt	Python package dependencies

```
pandas>=1.5.0  numpy>=1.23.0  scikit-learn>=1.2.0  tensorflow>=2.10.0
matplotlib>=3.6.0  seaborn>=0.12.0  plotly>=5.10.0  streamlit>=1.20.0  joblib>=1.2.0
```

Code 1: Requirements (requirements.txt)

```
1 import os, pickle, warnings
2 from pathlib import Path
3 import joblib, numpy as np, pandas as pd
4 import matplotlib.pyplot as plt, seaborn as sns
5 from sklearn.compose import ColumnTransformer
6 from sklearn.ensemble import (AdaBoostClassifier, BaggingClassifier,
7                               GradientBoostingClassifier, RandomForestClassifier)
8 from sklearn.impute import SimpleImputer
9 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
10 from sklearn.model_selection import train_test_split
11 from sklearn.pipeline import Pipeline
12 from sklearn.preprocessing import OneHotEncoder, StandardScaler
13 from sklearn.tree import DecisionTreeClassifier
14 warnings.filterwarnings("ignore")
15 sns.set_theme(style="whitegrid")
16
17 RANDOM_STATE = 42
18 SAMPLE_SIZE = 50_000
19
20 file_path = ("opt/Malware-Project/BigDataset/IoTScenarios/"
21             "CTU-IoT-Malware-Capture-1-1/bro/conn.log.labeled")
22
23 columns = [
24     "ts", "uid", "id.orig_h", "id.orig_p", "id.resp_h", "id.resp_p",
25     "proto", "service", "duration", "orig_bytes", "resp_bytes",
26     "conn_state", "local_orig", "local_resp", "missed_bytes", "history",
27     "orig_pkts", "orig_ip_bytes", "resp_pkts", "resp_ip_bytes", "label"
28 ]
29
30 df = pd.read_csv(
31     file_path, sep="\t", skiprows=8, names=columns,
32     nrows=SAMPLE_SIZE, na_values=["-", "", "(empty)"], low_memory=False,
33 )
34 print(f"Loaded shape: {df.shape}")
```

Code 2: Data Loading and Initial Setup (Notebook Cell 6)

```
1 def extract_traffic_type(value):
2     value = str(value)
3     if "Benign" in value:
```

```

4         return "Benign"
5     if "Malicious" in value:
6         return "Malicious"
7     return "Unknown"
8
9 df["traffic_type"] = df["label"].apply(extract_traffic_type)
10 df = df[df["traffic_type"].isin(["Benign", "Malicious"])]
11
12 numeric_cols = [
13     "ts", "id.orig_p", "id.resp_p", "duration", "orig_bytes", "resp_bytes",
14     "missed_bytes", "orig_pkts", "orig_ip_bytes", "resp_pkts", "resp_ip_bytes"
15 ]
16 for col in numeric_cols:
17     df[col] = pd.to_numeric(df[col], errors="coerce")
18
19 print(f"Cleaned shape: {df.shape}")
20 print(df["traffic_type"].value_counts())

```

Code 3: Target Variable Extraction and Numeric Coercion (Notebook Cell 8)

```

1 # Missing value analysis
2 missing_summary = (
3     df.isna().sum()
4     .to_frame("missing_count")
5     .assign(missing_percent=lambda x: 100 * x["missing_count"] / len(df))
6     .sort_values("missing_count", ascending=False)
7 )
8 display(missing_summary.head(15))
9
10 # Class distribution plot
11 plt.figure(figsize=(6, 4))
12 sns.countplot(data=df, x="traffic_type",
13               order=df["traffic_type"].value_counts().index)
14 plt.title("Traffic Type Distribution")
15 plt.show()
16
17 # Log-transformed numeric distributions
18 fig, axes = plt.subplots(1, 3, figsize=(16, 4))
19 for ax, col in zip(axes, ["duration", "orig_bytes", "resp_bytes"]):
20     sns.histplot(np.log1p(df[col].fillna(0)), bins=40, ax=ax)
21     ax.set_title(f"log1p({col})")
22 plt.tight_layout()
23 plt.show()
24
25 # Correlation heatmap
26 plt.figure(figsize=(12, 8))
27 sns.heatmap(df[numeric_cols].corr(numeric_only=True), cmap="coolwarm", center=0)
28 plt.title("Numeric Feature Correlation")
29 plt.show()

```

Code 4: Exploratory Data Analysis Visualizations (Notebook Cell 11)

```

1 def engineer_features(data):
2     data = data.copy()
3
4     # Missing value indicator flags -- strong anomaly signals in Zeek logs
5     for col in ["service", "history", "orig_bytes", "resp_bytes"]:
6         data[f"{col}_missing"] = data[col].isna().astype(int)
7
8     # Time-based features from Zeek epoch timestamp
9     ts = pd.to_datetime(data["ts"], unit="s", errors="coerce")
10    data["hour"] = ts.dt.hour
11    data["dayofweek"] = ts.dt.dayofweek
12
13    # Connection volume aggregate features
14    data["total_bytes"] = data["orig_bytes"].fillna(0) + data["resp_bytes"].fillna(0)
15    data["total_pkts"] = data["orig_pkts"].fillna(0) + data["resp_pkts"].fillna(0)
16    data["bytes_per_pkt"] = data["total_bytes"] / data["total_pkts"].replace(0, np.nan)

```

```

17
18 # Communication asymmetry ratios -- key for detecting malware C2 and exfiltration
19 data["orig_resp_byte_ratio"] = (data["orig_bytes"]
20                                / data["resp_bytes"].replace(0, np.nan))
21 data["orig_resp_pkt_ratio"] = (data["orig_pkts"]
22                                / data["resp_pkts"].replace(0, np.nan))
23
24 # Port behavior features
25 data["orig_port_is_well_known"] = (data["id.orig_p"] <= 1024).astype(int)
26 data["resp_port_is_well_known"] = (data["id.resp_p"] <= 1024).astype(int)
27
28 # Lightweight IP prefix features (avoid one-hot encoding full IPs)
29 data["orig_ip_prefix"] = (data["id.orig_h"].astype(str)
30                            .str.extract(r"^\d+\.\d+", expand=False))
31 data["resp_ip_prefix"] = (data["id.resp_h"].astype(str)
32                            .str.extract(r"^\d+\.\d+", expand=False))
33
34 # Drop identifiers and raw target fields
35 return data.drop(
36     columns=["uid", "label", "id.orig_h", "id.resp_h", "ts"], errors="ignore")
37
38 model_df = engineer_features(df)
39 X = model_df.drop(columns=["traffic_type"])
40 y = model_df["traffic_type"]
41
42 numeric_features = X.select_dtypes(include=["number", "bool"]).columns.tolist()
43 categorical_features = X.select_dtypes(include=["object", "category"]).columns.tolist()
44 print(f"Rows: {len(model_df)} | Numeric: {len(numeric_features)} | Cat: {len(
    categorical_features)}")

```

Code 5: Feature Engineering Function (Notebook Cell 14)

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y,
3 )
4 # Train shape: (40000, n_features) | Test shape: (10000, n_features)
5
6 try:
7     one_hot = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
8 except TypeError:
9     one_hot = OneHotEncoder(handle_unknown="ignore", sparse=False)
10
11 preprocessor = ColumnTransformer(
12     transformers=[
13         ("num", Pipeline(steps=[
14             ("imputer", SimpleImputer(strategy="median")),
15             ("scaler", StandardScaler()),
16         ]), numeric_features),
17         ("cat", Pipeline(steps=[
18             ("imputer", SimpleImputer(strategy="most_frequent")),
19             ("onehot", one_hot),
20         ]), categorical_features),
21     ],
22     remainder="drop",
23 )
24 print("Train shape:", X_train.shape, " | Test shape:", X_test.shape)

```

Code 6: Train-Test Split and Preprocessing Pipeline (Notebook Cell 16)

```

1 models = {
2     "Bagging - Decision Tree": BaggingClassifier(
3         estimator=DecisionTreeClassifier(random_state=RANDOM_STATE),
4         n_estimators=50, max_samples=0.80, n_jobs=-1,
5         random_state=RANDOM_STATE),
6     "Bagging - Random Forest": RandomForestClassifier(
7         n_estimators=150, max_depth=None, class_weight="balanced",
8         n_jobs=-1, random_state=RANDOM_STATE),
9     "Boosting - AdaBoost": AdaBoostClassifier(

```



```

10     n_estimators=100, learning_rate=0.5, random_state=RANDOM_STATE),
11     "Boosting - Gradient Boosting": GradientBoostingClassifier(
12         n_estimators=120, learning_rate=0.08, max_depth=3,
13         random_state=RANDOM_STATE),
14     "Boosting - Hist Gradient Boosting": HistGradientBoostingClassifier(
15         max_iter=120, learning_rate=0.08, random_state=RANDOM_STATE),
16 }
17
18 trained_models, results = {}, []
19 for name, estimator in models.items():
20     print(f"\nTraining: {name}")
21     pipeline = Pipeline(steps=[
22         ("preprocessor", preprocessor),
23         ("model", estimator),
24     ])
25     pipeline.fit(X_train, y_train)
26     y_pred = pipeline.predict(X_test)
27     accuracy = accuracy_score(y_test, y_pred)
28     trained_models[name] = pipeline
29     results.append({"model": name, "accuracy": accuracy})
30     print(f"Accuracy: {accuracy:.4f}")
31     print(classification_report(y_test, y_pred))
32
33 results_df = (pd.DataFrame(results)
34               .sort_values("accuracy", ascending=False)
35               .reset_index(drop=True))
36 display(results_df)

```

Code 7: Bagging and Boosting Model Training (Notebook Cell 18)

```

1 plt.figure(figsize=(10, 5))
2 sns.barplot(data=results_df, y="model", x="accuracy", palette="viridis")
3 plt.xlim(0, 1)
4 plt.title("Model Accuracy Comparison")
5 plt.show()
6
7 best_model_name = results_df.loc[0, "model"]
8 best_model = trained_models[best_model_name]
9 best_pred = best_model.predict(X_test)
10
11 cm = confusion_matrix(y_test, best_pred, labels=["Benign", "Malicious"])
12 plt.figure(figsize=(5, 4))
13 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
14             xticklabels=["Benign", "Malicious"],
15             yticklabels=["Benign", "Malicious"])
16 plt.title(f"Confusion Matrix: {best_model_name}")
17 plt.xlabel("Predicted"); plt.ylabel("Actual")
18 plt.show()
19 print(f"Best model: {best_model_name}")

```

Code 8: Confusion Matrix Visualization (Notebook Cell 19)

```

1 from sklearn.ensemble import IsolationForest, RandomForestClassifier
2 from sklearn.svm import OneClassSVM
3 from sklearn.metrics import (accuracy_score, precision_score,
4                               recall_score, f1_score, classification_report, confusion_matrix)
5 from tensorflow.keras.models import Model
6 from tensorflow.keras.layers import Input, Dense
7 from tensorflow.keras.callbacks import EarlyStopping
8
9 y_train_bin = y_train.map({"Benign": 0, "Malicious": 1}).astype(int)
10 y_test_bin = y_test.map({"Benign": 0, "Malicious": 1}).astype(int)
11
12 X_train_processed = preprocessor.fit_transform(X_train)
13 X_test_processed = preprocessor.transform(X_test)
14 if hasattr(X_train_processed, "toarray"):
15     X_train_processed = X_train_processed.toarray()
16     X_test_processed = X_test_processed.toarray()

```

```

17 X_train_processed = X_train_processed.astype("float32")
18 X_test_processed = X_test_processed.astype("float32")
19 X_train_benign = X_train_processed[y_train_bin.values == 0]
20
21 results, trained_models = [], {}
22
23 def evaluate_model(model_name, y_true, y_pred):
24     accuracy = accuracy_score(y_true, y_pred)
25     precision = precision_score(y_true, y_pred, zero_division=0)
26     recall = recall_score(y_true, y_pred, zero_division=0)
27     f1 = f1_score(y_true, y_pred, zero_division=0)
28     results.append({"model": model_name, "accuracy": accuracy,
29                    "precision": precision, "recall": recall, "f1_score": f1})
30     print(f"{model_name} | Acc:{accuracy:.4f} Prec:{precision:.4f}"
31           f" Recall:{recall:.4f} F1:{f1:.4f}")
32     print(classification_report(y_true, y_pred,
33                                target_names=["Benign", "Malicious"], zero_division=0))
34
35 # --- 1. Isolation Forest ---
36 contamination_rate = max(0.01, min(0.49, y_train_bin.mean()))
37 isolation_forest = IsolationForest(
38     n_estimators=150, contamination=contamination_rate,
39     random_state=RANDOM_STATE, n_jobs=-1)
40 isolation_forest.fit(X_train_benign)
41 iso_pred = np.where(isolation_forest.predict(X_test_processed) == -1, 1, 0)
42 trained_models["Isolation Forest"] = isolation_forest
43 evaluate_model("Isolation Forest", y_test_bin, iso_pred)
44
45 # --- 2. One-Class SVM (sampled for speed) ---
46 max_svm_samples = 10000
47 rng = np.random.default_rng(RANDOM_STATE)
48 X_svm_train = X_train_benign[
49     rng.choice(len(X_train_benign), size=min(max_svm_samples,
50         len(X_train_benign)), replace=False)]
51 one_class_svm = OneClassSVM(kernel="rbf", gamma="scale", nu=contamination_rate)
52 one_class_svm.fit(X_svm_train)
53 svm_pred = np.where(one_class_svm.predict(X_test_processed) == -1, 1, 0)
54 trained_models["One-Class SVM"] = one_class_svm
55 evaluate_model("One-Class SVM", y_test_bin, svm_pred)
56
57 # --- 3. Autoencoder ---
58 input_dim = X_train_processed.shape[1]
59 inp = Input(shape=(input_dim,))
60 enc = Dense(64, activation="relu")(inp)
61 enc = Dense(32, activation="relu")(enc)
62 enc = Dense(16, activation="relu")(enc)
63 dec = Dense(32, activation="relu")(enc)
64 dec = Dense(64, activation="relu")(dec)
65 out = Dense(input_dim, activation="linear")(dec)
66 autoencoder = Model(inputs=inp, outputs=out)
67 autoencoder.compile(optimizer="adam", loss="mse")
68 early_stop = EarlyStopping(monitor="val_loss", patience=3,
69                             restore_best_weights=True)
70 autoencoder.fit(X_train_benign, X_train_benign, epochs=20, batch_size=256,
71                 validation_split=0.1, callbacks=[early_stop], verbose=1)
72 train_mse = np.mean(np.square(X_train_benign -
73                               autoencoder.predict(X_train_benign, verbose=0)), axis=1)
74 threshold = np.percentile(train_mse, 95)
75 test_mse = np.mean(np.square(X_test_processed -
76                               autoencoder.predict(X_test_processed, verbose=0)), axis=1)
77 ae_pred = (test_mse > threshold).astype(int)
78 trained_models["Autoencoder"] = autoencoder
79 evaluate_model("Autoencoder", y_test_bin, ae_pred)
80
81 # --- 4. Random Forest (Supervised) ---
82 random_forest = Pipeline(steps=[
83     ("preprocessor", preprocessor),
84     ("model", RandomForestClassifier(

```

```

85         n_estimators=150, class_weight="balanced",
86         random_state=RANDOM_STATE, n_jobs=-1))]
87 random_forest.fit(X_train, y_train_bin)
88 rf_pred = random_forest.predict(X_test)
89 trained_models["Random Forest"] = random_forest
90 evaluate_model("Random Forest", y_test_bin, rf_pred)
91
92 comparison_df = (pd.DataFrame(results)
93                  .sort_values("f1_score", ascending=False)
94                  .reset_index(drop=True))
95 display(comparison_df)

```

Code 9: Anomaly Detection Models: Isolation Forest, One-Class SVM, Autoencoder, Random Forest (Notebook Cell 20)

```

1  # Save best model from independent evaluation
2  best_model_name = comparison_df.loc[0, "model"]
3  best_model      = trained_models[best_model_name]
4
5  if best_model_name == "Autoencoder":
6      best_model.save("best_network_anomaly_autoencoder.keras")
7      print("Best model saved as: best_network_anomaly_autoencoder.keras")
8  else:
9      joblib.dump(best_model, "best_network_anomaly_model.pkl")
10     print("Best model saved as: best_network_anomaly_model.pkl")
11 print(f"Best Model: {best_model_name}")
12
13 # Save also in organized model directory
14 model_dir = Path("models")
15 model_dir.mkdir(exist_ok=True)
16 joblib_path = model_dir / "best_iot23_ensemble_model.joblib"
17 pickle_path = model_dir / "best_iot23_ensemble_model.pkl"
18 metadata_path = model_dir / "model_results.csv"
19
20 joblib.dump(best_model, joblib_path)
21 with open(pickle_path, "wb") as f:
22     pickle.dump(best_model, f)
23 results_df.to_csv(metadata_path, index=False)
24 print(f"Saved to: {joblib_path} and {pickle_path}")

```

Code 10: Model Saving to .pkl and .joblib (Notebook Cell 20 & 22)

```

1  # Load the saved model and verify it produces the same results
2  loaded_model = joblib.load(joblib_path)
3  loaded_accuracy = accuracy_score(y_test, loaded_model.predict(X_test))
4  print(f"Reloaded model accuracy: {loaded_accuracy:.4f}")
5
6  # In Streamlit app.py -- model is loaded as:
7  # @st.cache_resource
8  # def load_model():
9  #     return joblib.load("best_network_anomaly_model.pkl")
10 # model = load_model()
11 # prediction = model.predict(engineered_input_features)

```

Code 11: Load Saved Model and Verify Accuracy (Notebook Cell 23)